

Migration to Cloud with Quarkus

Kacper Wereszko

22/09/2025 Gdańsk, Trójmiasto JUG



About me – Kacper Wereszko

- **Experience:**

- 12 years of professional experience in Java (Comarch, NIT, HL)
- 3 years of experience as team leader
- 3 years at HL KC
- 4 years of academic experience (PG, WSB)

- **Areas of interest:**

- **Professional:**

- Java (new features, Quarkus framework)
- Optimization algorithms and social network analysis
- Leadership in IT and Developers Experience

- **Private:**

- Fantasy
- Computer games (Dark Souls, Diablo, currently Silksong)
- Rollerblades



About the system – FIS

- **User base:**
 - Internal employees,
 - Contractors,
 - External parties (for some modules),
 - Located all over the world.
- **Migration process:**
 - Continuous transformation for roughly 10 years.
- **Target:**
 - Cloud-native,
 - Scalable,
 - Java/Jakarta EE based.

FIS 1: The Foundation

Platform: mainframe

Priority: stability

Provided a stable base for core enterprise operations.



FIS 2: The Monolith

Platform: Java-based

Priority: data accessibility

Improved the processes for accessing and managing data.

FIS 3: The Revolution

Platform: cloud-native and distributed

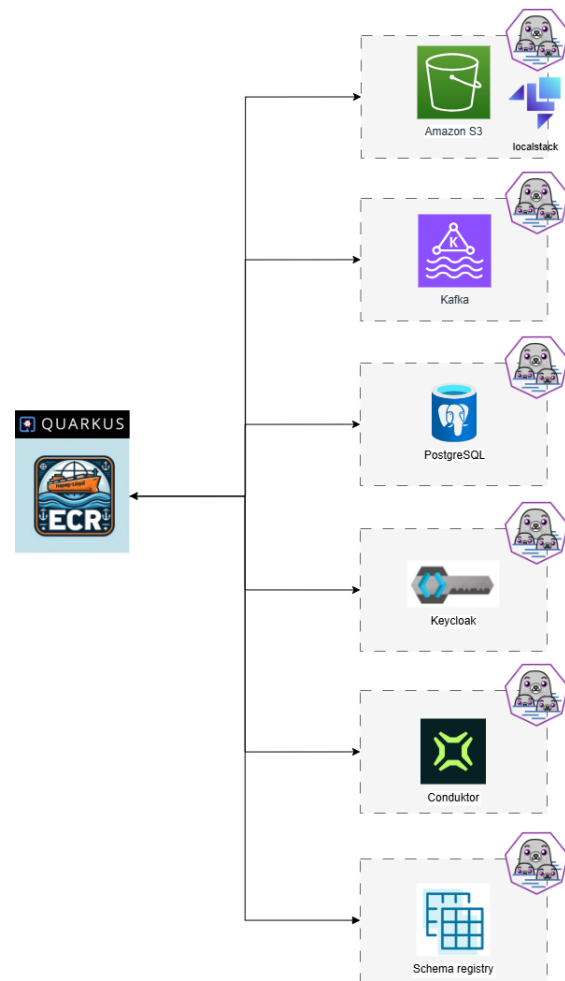
Priority: scalability and flexibility

A game-changing leap to future-ready enterprise systems.



FIS2 -> FIS3 migration strategy

1. Identify division into products and services
2. Design new UI
3. Rewrite app business logic to Java EE (still using common database)
4. Deploy services to cloud and move product-owned entities into product-owned DBs
 - Sync with monolith DB via CDC (Change Data Capture)
5. Introduce event-driven communication (Kafka)
6. Turn off the mainframe!
7. ...
8. Profit!



Why cloud?

Migration from monolith (mainframe) to microservices (cloud)

1. Scalability & availability
2. Cost optimization
3. Faster time-to-market
4. Deployment to multiple regions
5. Resilience & Fault Isolation
6. DevOps & better CI/CD Automation
7. Independent teams owning their services

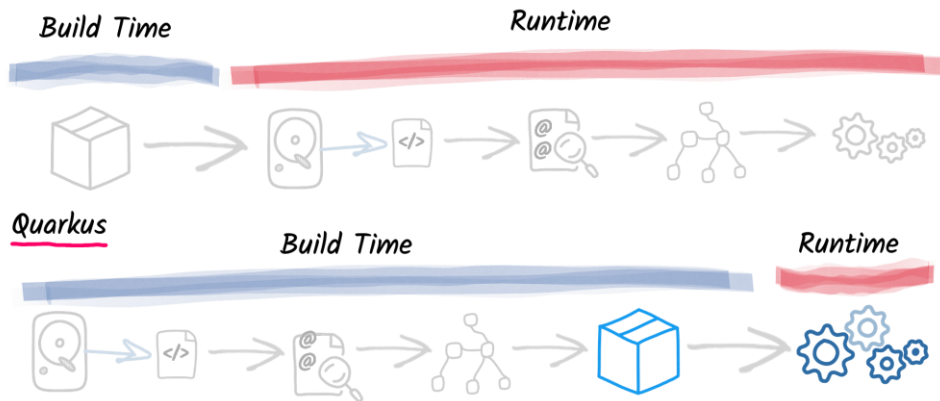


Why Quarkus?

- Proposed by architecture team in September 2024
- Quarkus turned out to be mature, stable and fast, with clear benefits:
 - Modernized devstack (from JDK8 to JDK21)
 - Enjoyable to work with
 - Integrated best practices
 - Fast and lightweight
 - Cloud ready (perfect for containers)
 - Shorter feedback loops
 - Community support



Traditional Frameworks



Quarkus features

- **Quarkus features:**
 - Dev Services, Testcontainers,
 - DEV UI,
 - Live coding (with live reload),
 - Continuous testing,
 - Simplified Hibernate ORM (Panache),
 - Good documentation at <https://quarkus.io/guides/>
- **JDK 21 features:**
 - Virtual Threads,
 - Records,
 - Improved APIs:
 - Stream API, Optional
 - Improved syntax:
 - Switch instruction, String literals, etc.



OpenJDK

01

Live demo

Developers insights

- **Good integration with Kafka,**
- **Convenient way of defining scheduled jobs,**
- **Compared to Spring: Proxy vs Interceptors,**
- **Live reload instead of „hot swaps”,**
- **Integration with .env files,**
- **Stable codebase (lack of deprecations),**
- **Supports Jakarta EE standards.**

Contact



KACPER WERESZKO

Team Leader (IT EQ ECR) Gdańsk

Hapag-Lloyd Knowledge Center Sp. z o.o.

Al. Grunwaldzka 413
80-309 Gdańsk

kacper.wereszko@hlag.com

www.hapag-lloyd.com

Thank you
for your attention!

